

# VendorPulse — Client Walkthrough (Shell)

## Speaker notes for the Solution Architecture diagram

*Purpose: a talk track for presenting the architecture to Shell. Every term is explained in plain English plus the "why Shell cares" angle. Read sections almost verbatim or paraphrase.*

### Opening

Say this first:

"This is a single-tenant application that runs entirely inside Shell's own Azure subscription, in your preferred IRM region. It spans five zones — your browser, a hardened edge, a private network holding the application, the supporting Azure services, and the handful of Microsoft 365 services it talks to. The theme to hold onto throughout: **the business logic is deterministic, the AI only drafts text behind a human-approval gate, everything internal stays private, and the only thing that ever leaves the network is Microsoft 365 — through one controlled egress.**"

Read it left to right = increasing trust zones: the user's browser on the left, Shell's most protected services on the right.

### 2.1 Solution Architecture — the 5 numbered zones (at a glance)

The architecture diagram is read left to right in 5 zones. Here is what each one means in plain language:

| # | Zone                                    | What it actually means                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Client                                  | The user's <b>web browser</b> . Three kinds of user — VMO Coordinator, Sponsor, Viewer — all log in with their normal Shell account ( <b>Entra SSO</b> ). Nothing is installed on their machine; roles are enforced on the server, not in the browser.                                                                                                                                                                                                                      |
| 2 | Edge                                    | <b>Azure Front Door + Application Gateway + WAF</b> — the secure front gate. Terminates HTTPS, applies a <b>Web Application Firewall</b> (blocks common web attacks), load-balances, and only lets traffic through to our network ("origin-lock"). The backend has no public address.                                                                                                                                                                                       |
| 3 | Shell Azure Subscription (Private VNet) | Shell's own slice of Azure, inside the approved region, holding a private network with <b>two Azure VMs</b> . <b>VM 1 (App Server)</b> runs the user-facing tier: the React 19 screen, login/permissions (Entra OIDC + RBAC), the 12-state <b>WorkflowEngine</b> , and the <b>approval gate</b> . <b>VM 2 (Backend Services)</b> runs the data/integration tier: the <b>PostgreSQL</b> database, the <b>GraphService</b> , and the <b>AI Service</b> that talks to Foundry. |

| # | Zone                | What it actually means                                                                                                                                                                                                                                                                                                                                        |
|---|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | Azure PaaS Services | The shared supporting services, reached privately and authenticated with a password-less <b>Managed Identity: Key Vault</b> (secrets & certificates), <b>Blob Storage</b> (minutes/transcripts), <b>App Insights + Log Analytics</b> (the OpenTelemetry audit trail), and <b>Azure AI Foundry (GPT-4o)</b> — the AI model, running inside Shell's own tenant. |
| 5 | External            | The only things reached outside, over outbound HTTPS through <b>Shell's egress proxy: Microsoft Graph</b> (send Outlook mail, manage calendars, create Teams meetings) and <b>Entra ID</b> (verify who the user is).                                                                                                                                          |

The sections below walk each zone in more depth, with the "why Shell cares" angle for each component.

## Zone 1 — Client (who uses it, and from what)

- Web client (browser)** — No app to install; everything runs in a standard browser. *Why Shell cares:* nothing to deploy to endpoints, smaller attack surface.
- VMO Coordinator / Sponsor / Viewer** — three roles: the Vendor Management Office coordinator who drives the work, the sponsor who approves, and read-only viewers. They sign in with their normal Shell identity via **Entra SSO** — no separate password. *Why Shell cares:* access is governed by Shell's existing identity, with role separation built in.
- OIDC + RBAC** — the browser handles the standard Entra sign-in redirect (OIDC); the user's role is carried in the token and enforced on the **backend**, never in the browser. *Why Shell cares:* security is enforced server-side; the UI only reflects what a role can see, it never decides it.

## Zone 2 — Edge (the front door and the bouncer)

- Azure Front Door + Application Gateway + WAF** — the single, hardened way in. Front Door is the global entry point and load balancer; Application Gateway is the regional gateway in front of the VNet; the **WAF** (Web Application Firewall) inspects and blocks attacks.
- TLS termination** — all traffic is encrypted (HTTPS).
- OWASP rule set** — the WAF blocks the common web attacks (SQL injection, cross-site scripting, etc.).
- Origin-lock** — the application only accepts traffic that arrives through this layer; you cannot bypass it and reach a VM directly.
- Load balancing / global routing** — traffic is distributed and routed reliably.
- Why Shell cares:* there is exactly one, inspected, hardened way in, and the backend has no public address.

## Zone 3 — Shell Azure Subscription / Private VNet (the application, all private)

Header: *Shell Azure Subscription — Private VNet, preferred IRM region.* This whole system is single-tenant inside one Shell-owned subscription. *Why Shell cares:* data never sits in a shared or multi-tenant environment, and it stays in the

approved region.

The application runs across **two Azure VMs** inside the private network — an application server and a backend-services server. Splitting them keeps the user-facing tier and the data/integration tier separate.

## Azure VM 1 — Application Server

- Entra OIDC auth + RBAC** — the login and authorization layer. Every API call must carry a valid Entra-issued token (JWT); the role/claims in that token decide what's allowed. *Why Shell cares:* authorization is enforced server-side, tied to Shell identity.
- Approval gate (HITL — Human-in-the-Loop)** (*the headline control, shown in red*) — every AI-generated item is produced as a **draft** and held here until a human approves or rejects. The workflow cannot move forward until a person acts. *Why Shell cares:* this is the direct implementation of Shell's "human oversight for privileged actions" control. The AI never acts on its own.
- WorkflowEngine (12-state)** — the heart of the system: a strict, forward-only state machine that drives a request from creation to completion across 12 states. This is deterministic — no AI. *Why Shell cares:* the business process cannot be skipped, reordered, or hallucinated; it is hard-coded logic.
- React 19 SPA** — the single-page web app, built from a shared **Design System**. The **ApprovalPanel** is the primary screen where a human reviews every AI draft. *Why Shell cares:* the human-approval control is a first-class part of the UI, not an afterthought.

## Azure VM 2 — Backend Services

- Azure PostgreSQL Flexible Server** — the primary relational database: workflow state, user data, approval records, audit logs. It is **VNet-private — no public endpoint**, reached over SSL with credentials drawn from Key Vault. *Why Shell cares:* the system's source of truth is a transactional, queryable, auditable store that is never exposed to the internet.
- GraphService** — the single component that talks to Microsoft 365 (mail, calendar, Teams). All its outbound calls go through the **Shell egress proxy**. *Why Shell cares:* one controlled integration point for all M365 actions.
- AI Service** — the only component that uses AI. It wraps calls to **Azure AI Foundry / GPT-4o** and only ever returns **drafted text** (summaries, suggestions). Outbound through the **egress proxy**. *Why Shell cares:* AI is boxed into one internal service, used only for text, behind the approval gate.

---

## Zone 4 — Azure PaaS Services (shared, supporting)

Consumed by both VMs, all reached privately, authenticated by **Managed Identity** (an Azure-issued identity — no stored passwords).

- Azure Key Vault** — the secrets safe: the LLM key, the Graph certificate, the JWT signing keys. *Why Shell cares:* no secrets in code or config files.
- Blob Storage** — stores meeting minutes and transcripts; treated as **immutable** once written. *Why Shell cares:* records of record can't be quietly altered.
- App Insights + Log Analytics** — observability and the immutable audit trail, instrumented with **OpenTelemetry (OTel)**. *Why Shell cares:* every action is traceable and the audit trail is tamper-evident.
- Azure AI Foundry (GPT-4o)** — the AI model service, running **inside Shell's own tenant**, used by the AI Service. *Why Shell cares:* the data sent to the model never leaves Shell's tenant — it is not a public API call.

---

## Zone 5 — External Services (the only things outside Shell's network)

All external calls leave through the **Shell egress proxy** — the single, governed exit point, allow-listed and logged. *Why Shell cares:* even if something inside were compromised, it cannot phone home to an arbitrary address.

**Microsoft Graph API (Shell tenant)** — Microsoft 365. The specific permissions used: **Mail.Send** (send invites), **Calendars.ReadWrite** (book meetings), **OnlineMeetings.ReadWrite** (create Teams calls). Authenticated with a **service-principal certificate** stored in Key Vault.

**Azure AD / Entra ID (Shell tenant)** — Shell's identity provider: SSO, group-to-role mapping, and the app's own identity (Managed Identity for service-to-service auth). *Why Shell cares*: these are standard Microsoft 365 services Shell already trusts and governs; nothing novel leaves the boundary.

## Data flow in one breath

"Browser signs in with Entra SSO → Front Door + WAF (TLS, OWASP) → Private VNet → VM 1 validates the token and runs the WorkflowEngine and the approval gate → VM 2 persists state to PostgreSQL and, when needed, calls Microsoft Graph and Azure AI Foundry through the egress proxy → secrets come from Key Vault, files from Blob, and every step is traced to App Insights. One private path in; the only thing leaving is Microsoft 365, through one controlled exit."

## The questions a Shell SA will ask — and your answers (rehearse these)

| Likely question                                  | Your answer                                                                                                                                                                                                                        |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Does our data leave Shell to reach the AI model? | No. Azure AI Foundry runs inside our tenant; the AI Service reaches it from inside the VNet. Data stays on our network.                                                                                                            |
| Can the AI take actions on its own?              | No. The AI Service only returns drafted text. Every action is held at the approval gate and only fires after a human approves.                                                                                                     |
| What is deterministic vs. AI?                    | All decisions — the 12-state workflow, approvals, persistence — are deterministic code. AI is used only to draft human-readable text.                                                                                              |
| How is it reachable — any public exposure?       | One way in: Front Door + Application Gateway + WAF, origin-locked. The VMs and database have no public endpoint.                                                                                                                   |
| How do you authenticate to Microsoft 365?        | A service-principal certificate stored in Key Vault — no passwords or shared secrets.                                                                                                                                              |
| Where are secrets?                               | Key Vault, fetched at runtime via Managed Identity — none in code or config.                                                                                                                                                       |
| Why a relational database, not just storage?     | It holds transactional workflow state, enforces integrity across related records, answers queries for dashboards, and gives a queryable audit trail — none of which object storage can do. (See the component-justification note.) |
| Can you prove what the AI did?                   | Every run is traced via OpenTelemetry to App Insights / Log Analytics, with an immutable audit trail.                                                                                                                              |
| Data residency?                                  | Single-tenant, in Shell's preferred IRM region.                                                                                                                                                                                    |

## One-line summary to leave them with

"VendorPulse is a single-tenant Shell-Azure application where the business logic is deterministic, AI is limited to drafting text behind a human approval gate, the AI model runs privately inside Shell's own tenant, and the only thing that ever leaves the network is Microsoft 365 — through one controlled, logged egress."